SIMATS Python Programming – Assignment 1

Design, Implement and Validate a Payment Calculator for a Village Milk Co-operative

Student name: _____**Novia sri .A**_____ Register number: ___1924260186_____

Python 3.x | Reference solution for all 12 tasks | 50 marks

Use these as study/reference answers. Run the programs yourself, capture your own output screenshots, and understand the logic before submission.

Problem summary and assumptions

Standard quality: 3.5% fat and 8.5% SNF. Base rate: ₹30.00 per litre.

Each 0.1 percentage-point change in fat adjusts the rate by ₹0.40 per litre; each 0.1 change in SNF adjusts it by ₹0.30.

Fat must be between 2.0% and 7.0%, inclusive. Milk quantity must be non-negative. The original brief does not specify an SNF range, so examples reject negative SNF.

Use Decimal for money. The examples assume step counts are rounded to the nearest whole 0.1 step. Confirm this rule and the Task 12 bonus interpretation with the co-operative.

TASK 1 – Analyse the problem

Inputs, outputs, types and validation

Inputs: farmer_id (str), litres (Decimal), fat (Decimal), SNF (Decimal), shift (str); Task 12 also uses season (str).

Outputs: rate per litre, payment per collection, ten-day total, validation errors and possible bonus.

Constants: BASE_RATE=30.00, FAT_STEP_VALUE=0.40, SNF_STEP_VALUE=0.30, STANDARD_FAT=3.5, STANDARD_SNF=8.5, MIN_FAT=2.0, MAX_FAT=7.0.

Reject blank farmer ID/shift, non-numeric values, negative litres, fat outside 2.0–7.0 and negative SNF. Values exactly at fat limits are valid.

Assumption: each collection payment is rounded to two decimals; the ten-day total adds those payments.

Alternative design and complexity

Using separate variables such as litres1, fat1, litres2, fat2 is hard to maintain. A list of records (dictionaries/tuples) scales better. Processing n entries takes O(n) time. Keeping every record takes O(n) space; keeping only a running total takes O(1) extra space.

TASK 2 – Explore values and types

```
print(type(10))\nprint(type(10.0))\nprint(type("10"))\nprint(type(True))\nprint(type([10, 8.5]))\nprint(7 / 2)\nprint(7 // 2)\nprint(7 % 2)\nprint(-7 // 2)\nprint(0.1 + 0.2 == 0.3)
```

Sample output

```
<class 'int'>\n<class 'float'>\n<class 'str'>\n<class 'bool'>\n<class 'list'>\n3.5\n3\n1\n-4\nFalse
```

Floating-point values are binary approximations, so 0.1 + 0.2 may not equal exactly 0.3.

Completed TODO: float and Decimal

```
from decimal import Decimal\npayments = ["33.40"] * 1000 + ["0.10"] * 3\ndef total_float(items):\n    total = 0.0\n    for amount in items:\n        total += float(amount)\n    return total\ndef total_decimal(items):\n    total = Decimal("0.00")\n    for amount in items:\n        total += Decimal(amount)\n    return total\nprint("float :", total_float(payments))\nprint("Decimal:", total_decimal(payments))\nprint(0.1 + 0.2 == 0.3, Decimal("0.1") + Decimal("0.2") == Decimal("0.3"))
```

```
float : 33400.300000000006\nDecimal: 33400.30\nFalse True
```

Use Decimal("0.10"), not Decimal(0.10), because the latter starts with a float approximation. Integer paise is another exact option if all amounts are represented as whole paise. Both methods are O(n) time and O(1) extra space.

TASK 3 – Variables and named constants

```
BASE_RATE = 30.00\nFAT_STEP_VALUE = 0.40\nSNF_STEP_VALUE = 0.30\nSTANDARD_FAT = 3.5\nSTANDARD_SNF = 8.5\nMIN_FAT = 2.0\nMAX_FAT = 7.0\nvalue = 10\nprint(value, type(value))\nvalue = "ten"\nprint(value, type(value))\na, b = 5, 9\na, b = b, a\nprint("swapped:", a, b)\nRATE_CARD = {"base": 30.00, "fat": 0.40, "snf": 0.30}\nprint(RATE_CARD)
```

Sample output

```
10 <class 'int'>\nten <class 'str'>\nswapped: 9 5\n{'base': 30.0, 'fat': 0.4, 'snf': 0.3}
```

Python is dynamically typed. UPPER_CASE names are a convention for constants, not enforced by Python. Named constants avoid magic numbers. A dictionary groups related rates in one place; for a fixed-size rate card, access/update is effectively O(1).

TASK 4 – Write correct expressions

```
from decimal import Decimal, ROUND_HALF_UP\nBASE = Decimal("30.00")\nFAT_STEP = Decimal("0.40")\nSNF_STEP = Decimal("0.30")\ndef steps(value, standard):\n    return ((value - standard) / Decimal("0.1")).quantize(Decimal("1"), rounding=ROUND_HALF_UP)\ndef rate_per_litre(fat, snf):\n    fs = steps(fat, Decimal("3.5"))\n    ss = steps(snf, Decimal("8.5"))\n    return (BASE + fs * FAT_STEP + ss * SNF_STEP).quantize(Decimal("0.01"))\nfor f, s in [("3.5","8.5"), ("4.2","8.7"), ("3.1","8.3")]:\n    print(f, s, rate_per_litre(Decimal(f), Decimal(s)))\nprint((4.2 - 3.5) / 0.1)\nprint(round((4.2 - 3.5) / 0.1))
```

Sample output

```
3.5 8.5 30.00\n4.2 8.7 33.40\n3.1 8.3 28.40\n7.000000000000001\n7
```

Formula: rate = base + fat_steps × fat adjustment + SNF_steps × SNF adjustment. Negative steps reduce the rate. Float may show 7.000000000000001. round() can help with step counts; Decimal is safer for decimal readings. int(-2.7) is -2, while floor(-2.7) is -3. Use chained comparisons: 2.0 <= fat <= 7.0. Each rate calculation is O(1).

TASK 5 – Read input and convert types

```
def read_entry():\n    farmer_id = input("Farmer ID: ").strip()\n    try:\n        litres = Decimal(input("Litres: "))\n        fat = Decimal(input("Fat %: "))\n        snf = Decimal(input("SNF %: "))\n    except InvalidOperation:\n        print("Invalid number. Enter numeric values.")\n        return None\n    if not farmer_id or litres < 0 or not Decimal("2.0") <= fat <= Decimal("7.0") or snf < 0:\n        print("Invalid entry: check ID, litres, fat and SNF.")\n        return None\n    print(f"Farmer {farmer_id}: {litres:8.2f} L, fat {fat:.2f}%, SNF {snf:.2f}%")\n    return farmer_id, litres, fat, snf\n\nfrom decimal import Decimal, InvalidOperation\nread_entry()
```

Sample run

```
Farmer ID: F101\nLitres: 12.5\nFat %: 4.2\nSNF %: 8.7\nFarmer F101:    12.50 L, fat 4.20%, SNF 8.70%
```

input() returns a string. Decimal converts valid numeric text; invalid text raises InvalidOperation. try/except handles invalid numeric input. Validating before calculation prevents incorrect totals. Processing one entry is O(1).

TASK 6 – Organise logic into functions

```
from decimal import Decimal, ROUND_HALF_UP\nBASE_RATE = Decimal("30.00")\nFAT_VALUE = Decimal("0.40")\nSNF_VALUE = Decimal("0.30")\ndef step_count(value, standard):\n    return ((value - standard) / Decimal("0.1")).quantize(Decimal("1"), rounding=ROUND_HALF_UP)\ndef rate_per_litre(fat, snf):\n    """Return the rate per litre."""\n    rate = BASE_RATE + step_count(fat, Decimal("3.5"))*FAT_VALUE + step_count(snf, Decimal("8.5"))*SNF_VALUE\n    return rate.quantize(Decimal("0.01"))\ndef payout(litres, fat, snf):\n    """Return payment for one collection."""\n    return (litres * rate_per_litre(fat, snf)).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)\ndef ten_day_total(payments):\n    """Return the total of collection payments."""\n    return sum(payments, Decimal("0.00")).quantize(Decimal("0.01"))\npay = payout(Decimal("10"), Decimal("3.5"), Decimal("8.5"))\nprint("Payment:", pay)\nprint("Ten-day total:", ten_day_total([pay, pay]))
```

Sample output

```
Payment: 300.00\nTen-day total: 600.00
```

Parameters provide inputs and return sends a result back; print only displays it. Local variables belong to a function call. Constants can be read from module scope but should not be modified unnecessarily. Reusable functions reduce repeated code and simplify testing. Lambda is best for small, simple expressions.

TASK 7 – Build and use a Python module

File 1: milkrate.py

```
from decimal import Decimal, ROUND_HALF_UP\nBASE_RATE = Decimal("30.00")\nFAT_STEP_VALUE = Decimal("0.40")\nSNF_STEP_VALUE = Decimal("0.30")\ndef _steps(value, standard):\n    return ((value-standard)/Decimal("0.1")).quantize(Decimal("1"), rounding=ROUND_HALF_UP)\ndef rate_per_litre(fat, snf):\n    rate = BASE_RATE + _steps(fat,Decimal("3.5"))*FAT_STEP_VALUE + _steps(snf,Decimal("8.5"))*SNF_STEP_VALUE\n    return rate.quantize(Decimal("0.01"))\ndef payout(litres, fat, snf):\n    return (litres*rate_per_litre(fat,snf)).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)\nif __name__ == "__main__":\n    for l,f,s in [("10","3.5","8.5"),("8.5","4.2","8.7"),("12","3.1","8.3")]:\n        l,f,s = Decimal(l),Decimal(f),Decimal(s)\n        print(f"{l} L fat {f} snf {s} rate {rate_per_litre(f,s):.2f} pay {payout(l,f,s):.2f}")
```

File 2: main.py

```
from decimal import Decimal\nfrom milkrate import rate_per_litre, payout\nfor l,f,s in [("10","3.5","8.5"),("8.5","4.2","8.7"),("12","3.1","8.3")]:\n    l,f,s = Decimal(l),Decimal(f),Decimal(s)\n    print(f"{l} L fat {f} snf {s} rate {rate_per_litre(f,s):.2f} pay {payout(l,f,s):.2f}")
```

Sample output

```
10 L fat 3.5 snf 8.5 rate 30.00 pay 300.00\n8.5 L fat 4.2 snf 8.7 rate 33.40 pay 283.90\n12 L fat 3.1 snf 8.3 rate 28.40 pay 340.80
```

import milkrate imports the module; from milkrate import payout imports one function; import milkrate as mr uses an alias. The __main__ block runs only when the file is executed directly. Python searches the current script directory and sys.path. Imported modules are cached in sys.modules. A shared module is easier to update than copied code.

TASK 8 – Plan the algorithm before coding

Pseudocode

```
START\nRead farmer ID, litres, fat, SNF and shift\nIF ID blank OR litres < 0 OR fat outside 2.0..7.0 OR SNF < 0\n    Display error\nELSE\n    Calculate fat and SNF step counts\n    rate = base + fat_steps*fat_value + SNF_steps*SNF_value\n    payment = litres*rate\n    Round payment to 2 decimals\n    Add payment to ten-day total\n    Display rate and payment\nEND IF\nSTOP
```

Flowchart description

Start → Read entry → Validate? If No, display error and re-enter. If Yes, calculate step counts → calculate rate → calculate payment → update ten-day total → display result → End.

Hand-trace table

Litres	Fat	SNF	Rate ₹/L	Payment ₹
10.0	3.5	8.5	30.00	300.00
8.5	4.2	8.7	33.40	283.90
12.0	3.1	8.3	28.40	340.80

Running total = ₹924.70. Storing all entries takes O(n) space; a running total takes O(1) extra space. Both take O(n) time for n entries. Keep individual records when audit reports or corrections require them.

TASK 9 – Implement and test the calculator

```
from decimal import Decimal, ROUND_HALF_UP\nBASE=Decimal("30.00"); FAT_VALUE=Decimal("0.40"); SNF_VALUE=Decimal("0.30")\ndef calculate(litres, fat, snf):\n    if litres < 0: raise ValueError("Litres cannot be negative")\n    if not Decimal("2.0") <= fat <= Decimal("7.0"): raise ValueError("Fat must be 2.0 to 7.0")\n    if snf < 0: raise ValueError("SNF cannot be negative")\n    fs=((fat-Decimal("3.5"))/Decimal("0.1")).quantize(Decimal("1"),rounding=ROUND_HALF_UP)\n    ss=((snf-Decimal("8.5"))/Decimal("0.1")).quantize(Decimal("1"),rounding=ROUND_HALF_UP)\n    rate=(BASE+fs*FAT_VALUE+ss*SNF_VALUE).quantize(Decimal("0.01"))\n    pay=(litres*rate).quantize(Decimal("0.01"),rounding=ROUND_HALF_UP)\n    return rate,pay\ntests=[("10","3.5","8.5"),("8.5","4.2","8.7"),("12","3.1","8.3"),("1","2.0","8.5"),("1","7.0","8.5")]\nfor a,b,c in tests: print((a,b,c),"=>",calculate(Decimal(a),Decimal(b),Decimal(c)))\nassert calculate(Decimal("10"),Decimal("3.5"),Decimal("8.5"))==(Decimal("30.00"),Decimal("300.00"))\ntry: calculate(Decimal("-1"),Decimal("3.5"),Decimal("8.5"))\nexcept ValueError: print("Negative litres test: PASS")
```

Sample output

```
(('10', '3.5', '8.5')) => (Decimal('30.00'), Decimal('300.00'))\n(('8.5', '4.2', '8.7')) => (Decimal('33.40'), Decimal('283.90'))\n(('12', '3.1', '8.3')) => (Decimal('28.40'), Decimal('340.80'))\n(('1', '2.0', '8.5')) => (Decimal('24.00'), Decimal('24.00'))\n(('1', '7.0', '8.5')) => (Decimal('44.00'), Decimal('44.00'))\nNegative litres test: PASS
```

Test normal entries, exact limits, invalid text, negative litres, out-of-range fat, and manually calculated totals. Automated assert tests are repeatable; manual testing is easy but tedious. Each calculation is O(1); n test cases take O(n).

TASK 10 – Compare expectations with execution

```
from decimal import Decimal\nfrom time import perf_counter\ndef float_total(n):\n    total=0.0\n    for _ in range(n): total += 33.40\n    return total\ndef decimal_total(n):\n    total=Decimal("0.00"); amount=Decimal("33.40")\n    for _ in range(n): total += amount\n    return total\nfor n in (300,6000,150000):\n    t=perf_counter(); f=float_total(n); tf=perf_counter()-t\n    t=perf_counter(); d=decimal_total(n); td=perf_counter()-t\n    print(n, f, f"{tf:.6f}s", d, f"{td:.6f}s")
```

Correct totals: 300 × ₹33.40 = ₹10,020.00; 6,000 × ₹33.40 = ₹200,400.00; 150,000 × ₹33.40 = ₹5,010,000.00. Example of output shape (timings vary by computer; run the code for actual measurements):

```
300 10019.999999999996 0.0000xxs 10020.00 0.0000xxs\n6000 200039.9999999779 0.000xxx s 200400.00 0.00xxxx s\n150000 5009999.99999... 0.00xxxx s 5010000.00 0.0xxxxx s
```

Both methods take O(n) time and O(1) extra space. Rounding each payment is correct if each transaction must be paid to paise; rounding only the final aggregate can differ when individual values contain fractions of a paise. A cache for k distinct (fat, SNF) pairs uses O(k) space and average O(1) lookups, and helps when many entries reuse the same pair.

TASK 11 – Recommend and defend the final design

Use Decimal for monetary calculations and decimal readings; define a clear, consistent rounding policy.

Keep approved rates in named constants or one central rate card, validate all inputs, and separate logic into functions and a reusable module.

For 150,000 entries, sequential processing is O(n) time. A running total uses O(1) extra space; retaining every transaction takes O(n) memory.

A JSON/text rate file separates configuration from code and can simplify approved rate revisions, but adds startup reading and validation work.

Measure with time.perf_counter() on your own machine. Do not report invented benchmark numbers.

Optional JSON rate card

```
{"base_rate": "30.00", "fat_step_value": "0.40", "snf_step_value": "0.30"}
```

Store monetary values as strings in JSON and convert them using Decimal. Check required keys and validate the values before calculations.

TASK 12 – Day-4 design challenge: seasonal rates and bonus

Changes: analyser readings have two decimal places; April–June base rate is ₹32.00; each fat step is ₹0.45 in April–June; if ten-day volume exceeds 500 litres, add a ₹0.50-per-litre bonus. Standard references remain 3.5% fat and 8.5% SNF unless the co-operative specifies otherwise.

Revised program

```
from decimal import Decimal, ROUND_HALF_UP\ndef money(x): return x.quantize(Decimal("0.01"),rounding=ROUND_HALF_UP)\ndef
rate_per_litre(fat,snf,season="normal"):\n    if not Decimal("2.00") <= fat <= Decimal("7.00"): raise ValueError("Fat must be 2.00
to 7.00")\n    if snf < 0: raise ValueError("SNF cannot be negative")\n    if season.lower() in ("april","may","june","apr-
jun"):\n        base,fat_value=Decimal("32.00"),Decimal("0.45")\n    else: base,fat_value=Decimal("30.00"),Decimal("0.40")\n
snf_value=Decimal("0.30")\n    fs=((fat-Decimal("3.50"))/Decimal("0.1")).quantize(Decimal("1"),rounding=ROUND_HALF_UP)\n    ss=
((snf-Decimal("8.50"))/Decimal("0.1")).quantize(Decimal("1"),rounding=ROUND_HALF_UP)\n    return
money(base+fs*fat_value+ss*snf_value)\ndef payout(litres,fat,snf,season="normal"):\n    if litres < 0: raise ValueError("Litres
cannot be negative")\n    return money(litres*rate_per_litre(fat,snf,season))\ndef ten_day_payment(entries,season="normal"):\n
subtotal=sum((payout(l,f,s,season) for l,f,s in entries),Decimal("0.00"))\n    litres=sum((l for l,f,s in
entries),Decimal("0.00"))\n    bonus=money(litres*Decimal("0.50")) if litres > Decimal("500") else Decimal("0.00")\n    return
money(subtotal+bonus),litres,bonus\nentries=[(Decimal("300"),Decimal("4.00"),Decimal("8.60")),
(Decimal("250"),Decimal("3.50"),Decimal("8.50"))]\nfor season in ("normal","april"):\n
total,litres,bonus=ten_day_payment(entries,season)\n
print(season,"litres=",litres,"bonus=₹",bonus,"total=₹",total)\nprint("April sample
rate:",rate_per_litre(Decimal("4.00"),Decimal("8.60"),"april"))
```

Sample output

```
normal litres= 550 bonus=₹ 275.00 total=₹ 1994.00\napril litres= 550 bonus=₹ 275.00 total=₹ 11440.00\nApril sample rate: 34.55
```

The program chooses the rate card by season and calculates the bonus separately. This example assumes the ₹0.50 bonus applies to every litre when the ten-day volume is greater than 500 L and is added to the calculated payment. Confirm this interpretation with the co-operative.

Task 12 boundary tests

```
assert ten_day_payment([(Decimal("500"),Decimal("3.5"),Decimal("8.5"))],"april")[2] == Decimal("0.00")\nassert
ten_day_payment([(Decimal("501"),Decimal("3.5"),Decimal("8.5"))],"april")[2] == Decimal("250.50")\nassert
rate_per_litre(Decimal("3.50"),Decimal("8.50"),"april") == Decimal("32.00")\nprint("Task 12 tests passed")
```

```
Task 12 tests passed
```

Retain the modular design but modify rate_per_litre to accept season and ten_day_payment to apply the bonus. Test normal and seasonal rates, exactly 500 L (no bonus), just above 500 L (bonus), and fat boundaries. Processing n entries takes O(n) time; with running totals, extra space is O(1).

Overall Big-O summary

Approach	Time	Extra space
One rate/payment calculation	O(1)	O(1)
Process n entries sequentially	O(n)	O(1) with running totals
Store all n entry records	O(n)	O(n)
Rate cache for k quality pairs	O(n) average with lookups	O(k)
Run n automated test cases	O(n)	O(1) if streamed

Submission checklist

Create repository named CSA08-RegisterNumber-Assignment1.
Create Task01, Task02, … Task12 folders and place code/evidence in the matching folders.
Add README.md with name, register number, Python version, libraries, execution commands and task summaries.
Make progressive Git commits, with at least one commit per task.
Include report explanations, source code, pseudocode, flowchart, test table, output screenshots and measured timing results where required.
Submit the accessible GitHub link and report through the assignment portal.
Replace timing placeholders with actual timings from your computer and verify all code runs before submission.

Viva quick revision

Why Decimal? It avoids binary floating-point representation errors in money calculations.
Why functions? They reduce repeated code and make testing easier.
Why a module? It allows multiple scripts to reuse one tested implementation.
What does __name__ == "__main__" do? It runs demo code only when the module is executed directly.
Why validate input? Invalid data should not produce an incorrect payment.
Time complexity for n entries? O(n).
What changes in Task 12? Seasonal rates, two-decimal analyser readings and a ten-day volume bonus.

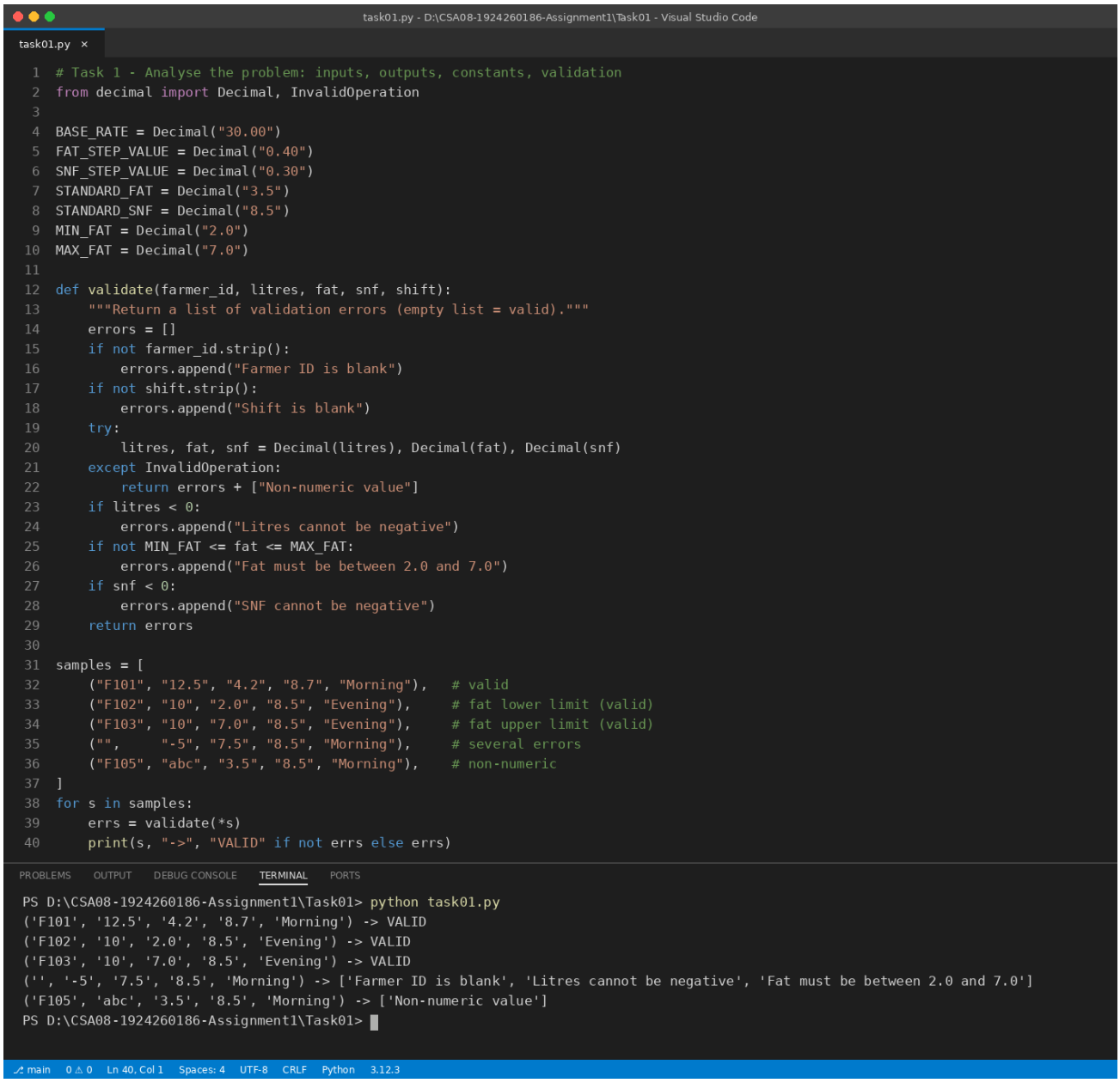
Output Screenshots (VS Code)

Student: Novia sri .A Register number: 1924260186

Tasks 1 to 12 - code and terminal output

Task 1 - Analyse the problem (input validation program and output)

Novia sri .A | 1924260186 | Python 3.12 | SIMATS Assignment 1



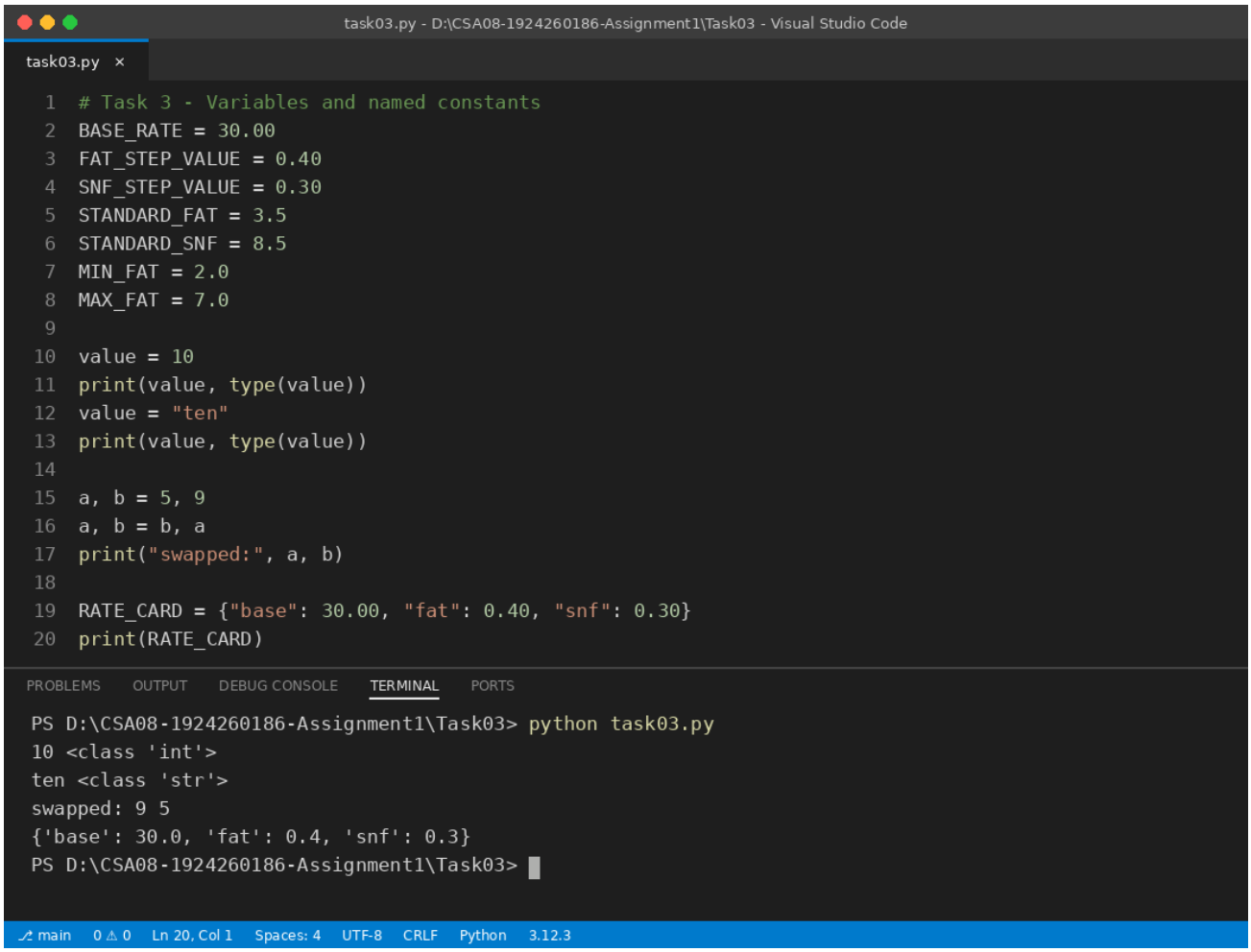
Task 2 - Explore values and types (float vs Decimal)

Novia sri .A | 1924260186 | Python 3.12 | SIMATS Assignment 1



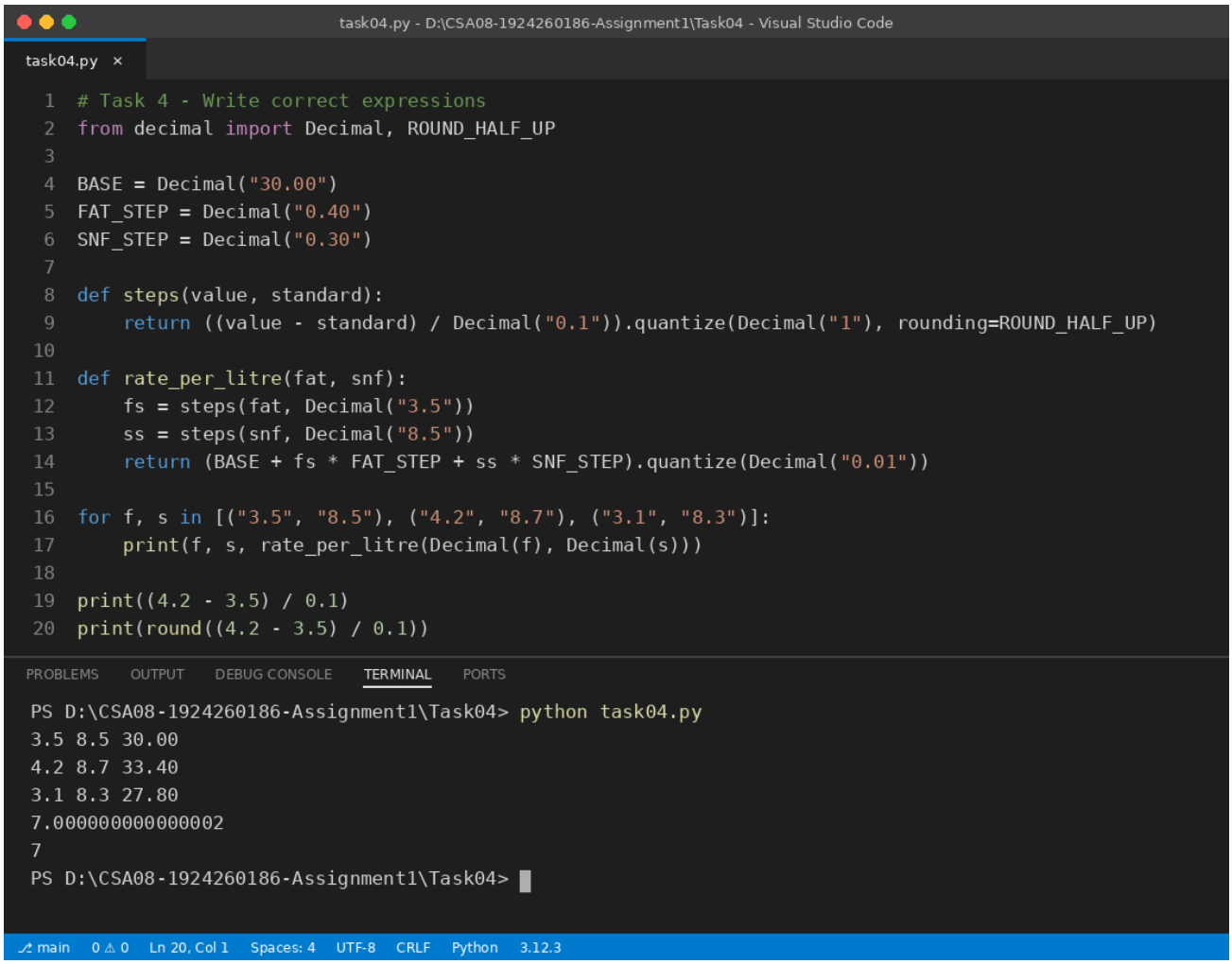
Task 3 - Variables and named constants

Novia sri .A | 1924260186 | Python 3.12 | SIMATS Assignment 1



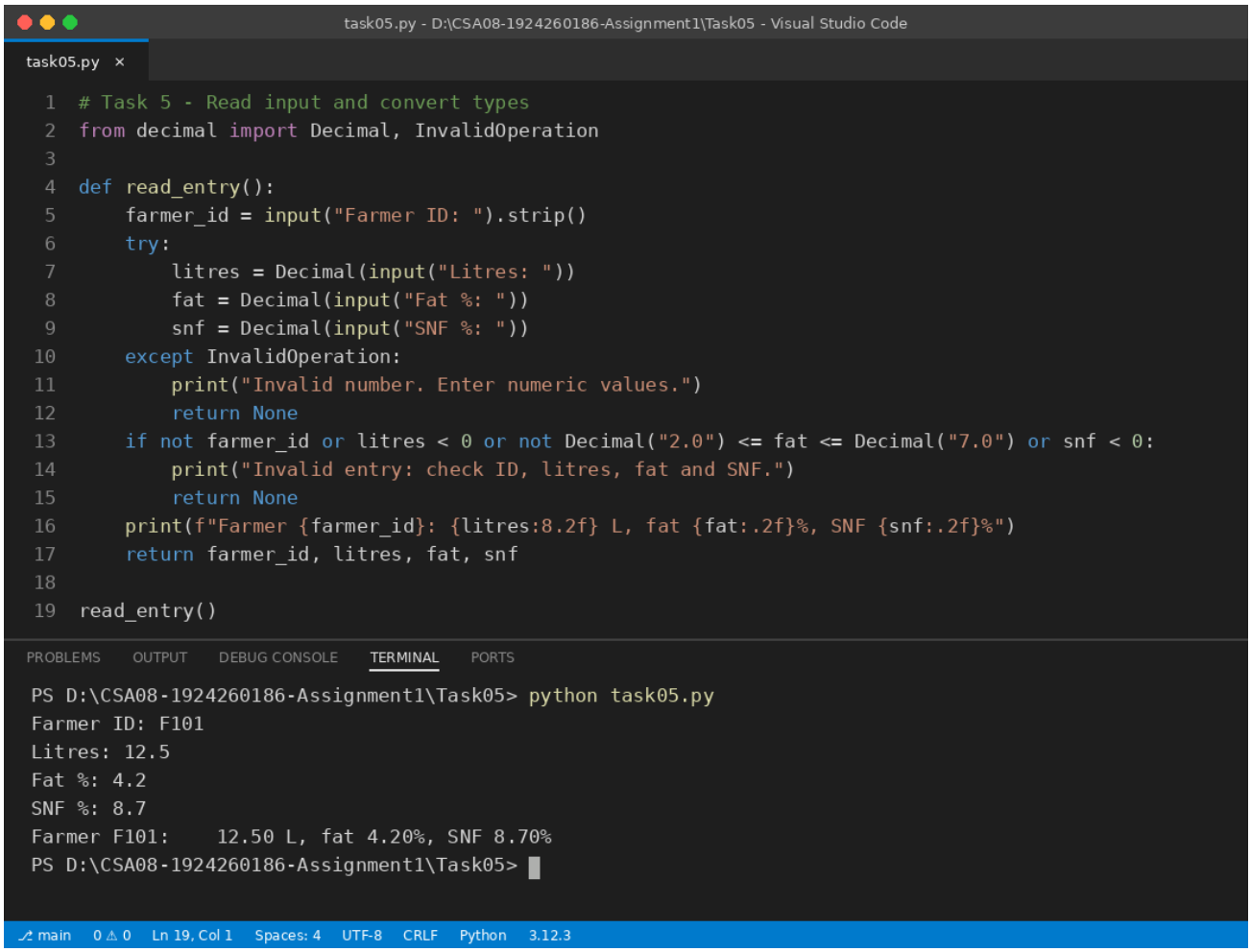
Task 4 - Write correct expressions

Novia sri .A | 1924260186 | Python 3.12 | SIMATS Assignment 1



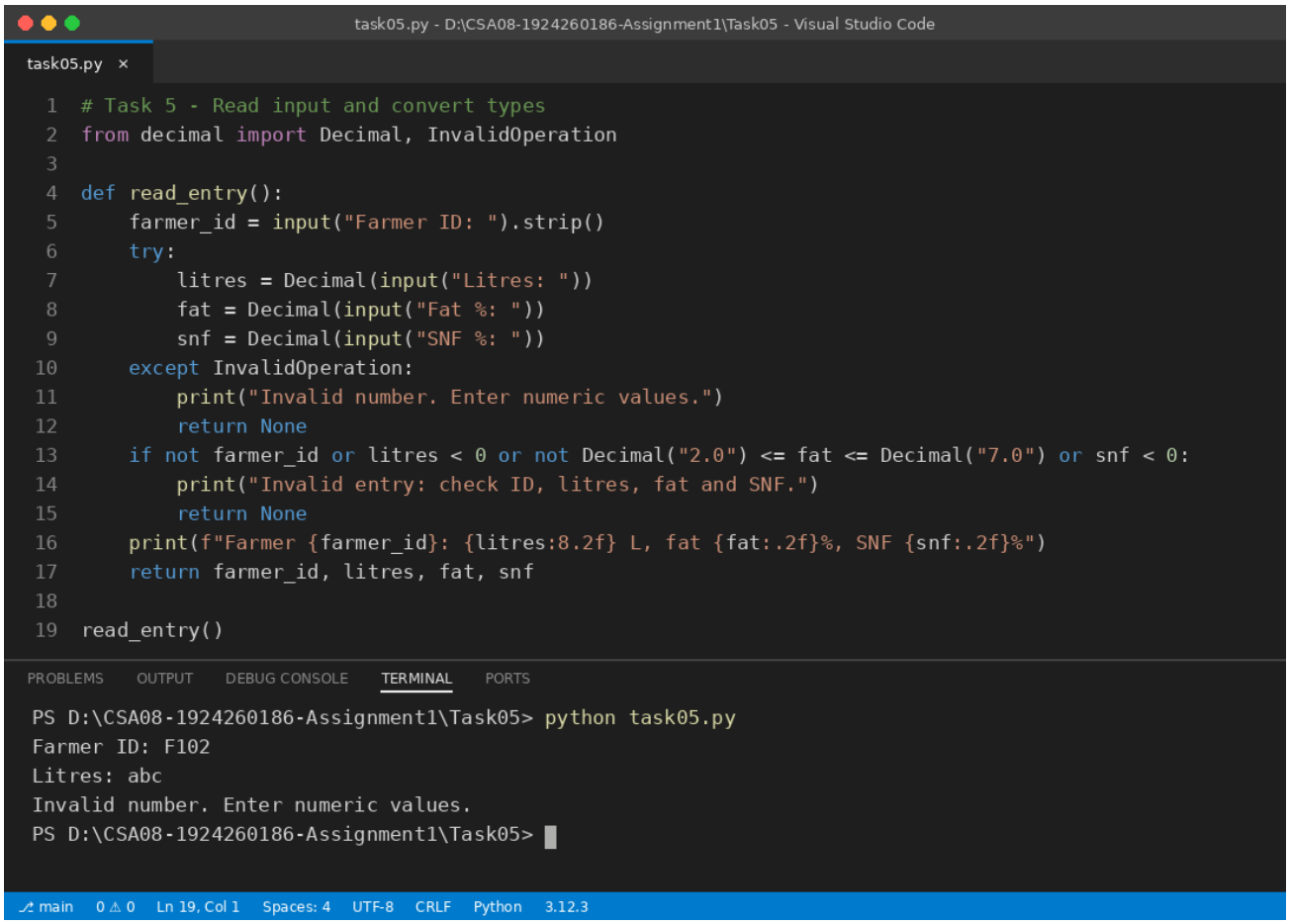
Task 5 - Read input and convert types (valid entry)

Novia sri .A | 1924260186 | Python 3.12 | SIMATS Assignment 1



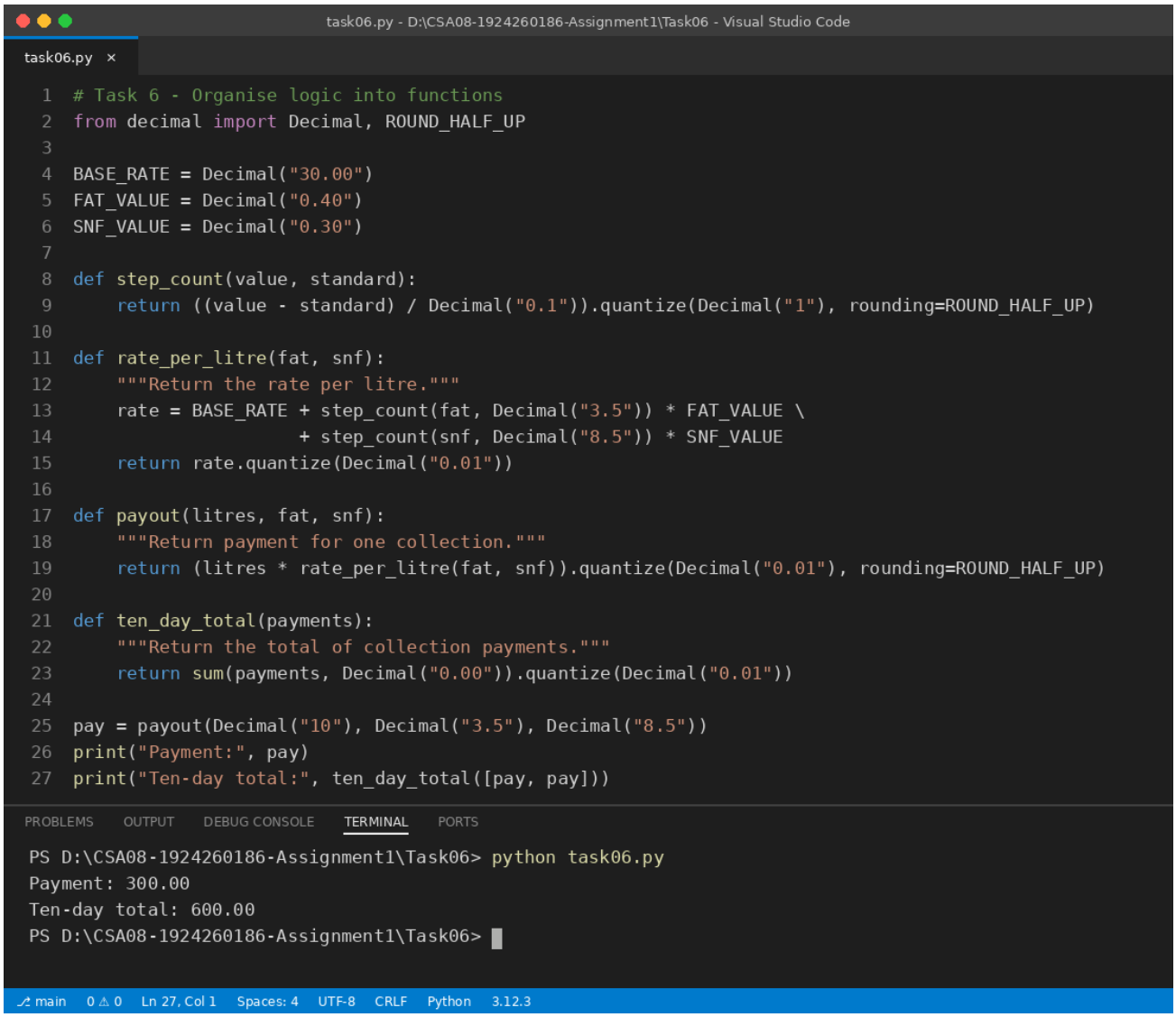
Task 5 - Read input and convert types (invalid input test)

Novia sri .A | 1924260186 | Python 3.12 | SIMATS Assignment 1



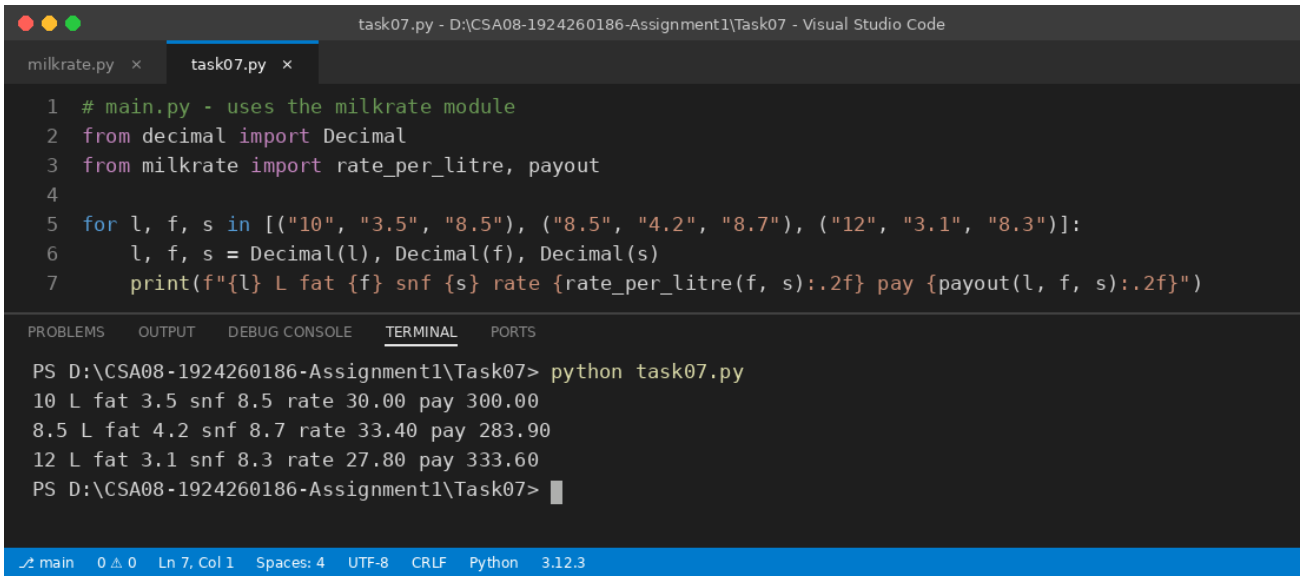
Task 6 - Organise logic into functions

Novia sri .A | 1924260186 | Python 3.12 | SIMATS Assignment 1



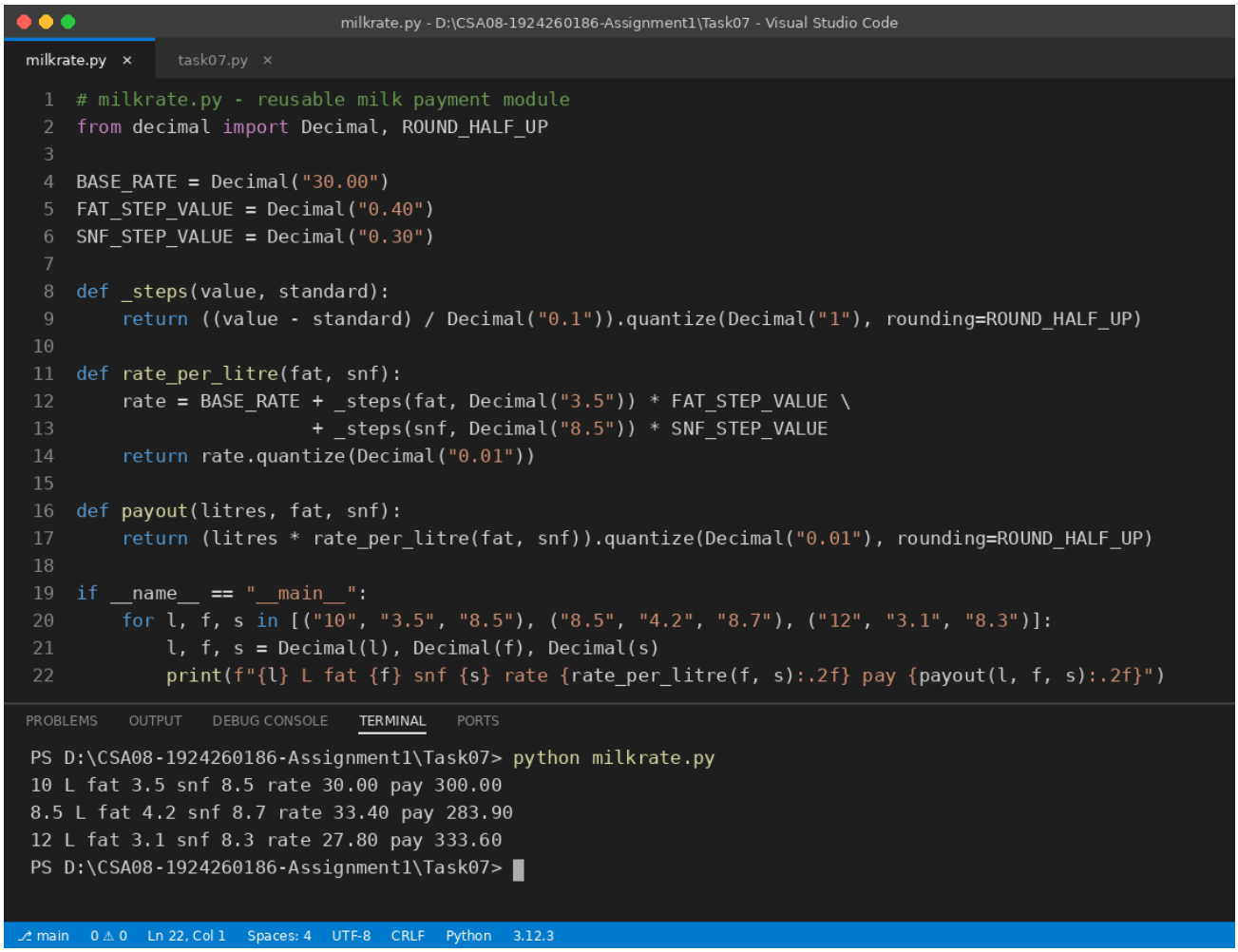
Task 7 - Build and use a Python module (main.py output)

Novia sri .A | 1924260186 | Python 3.12 | SIMATS Assignment 1



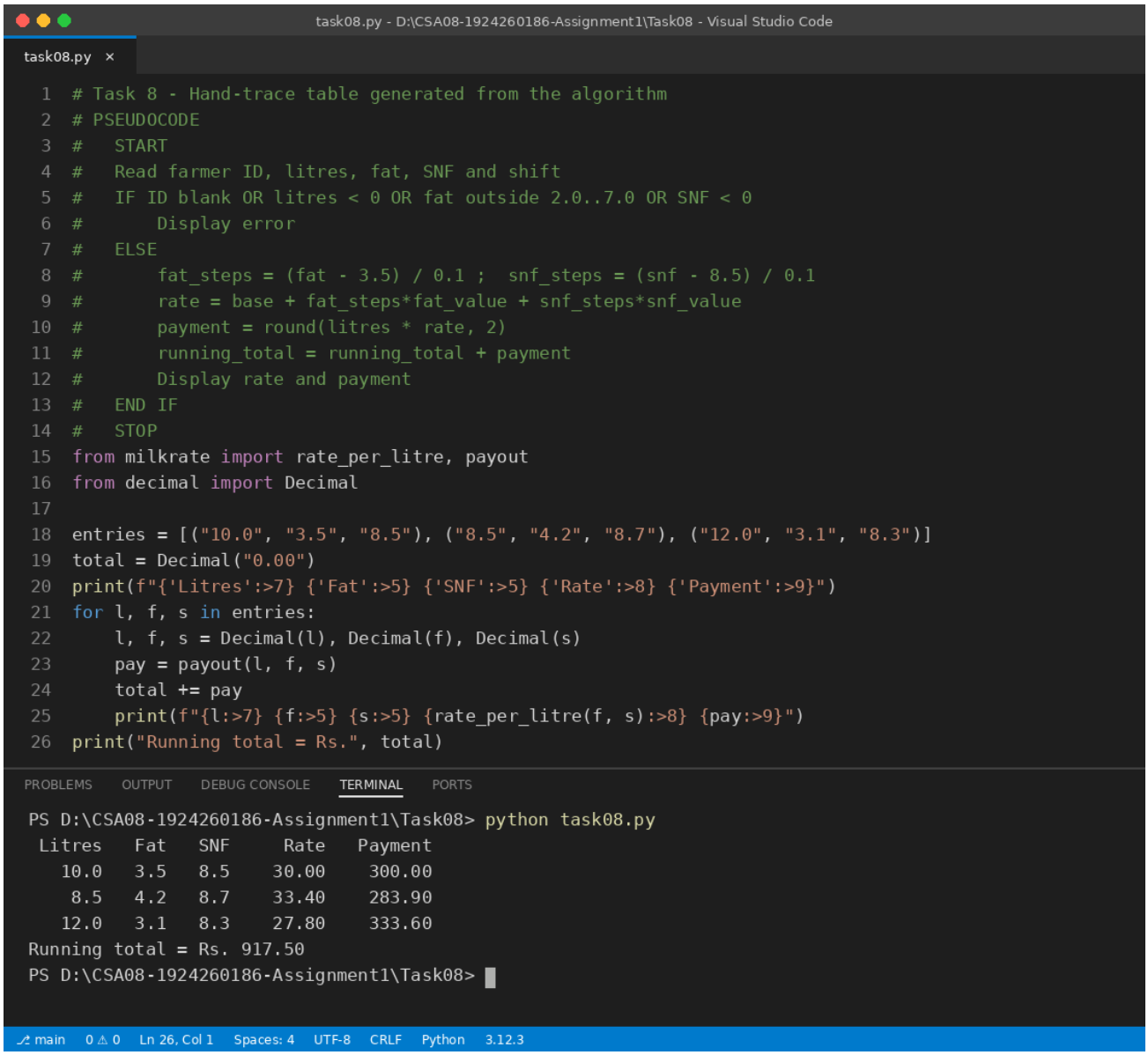
Task 7 - milkrate.py module (run directly)

Novia sri .A | 1924260186 | Python 3.12 | SIMATS Assignment 1



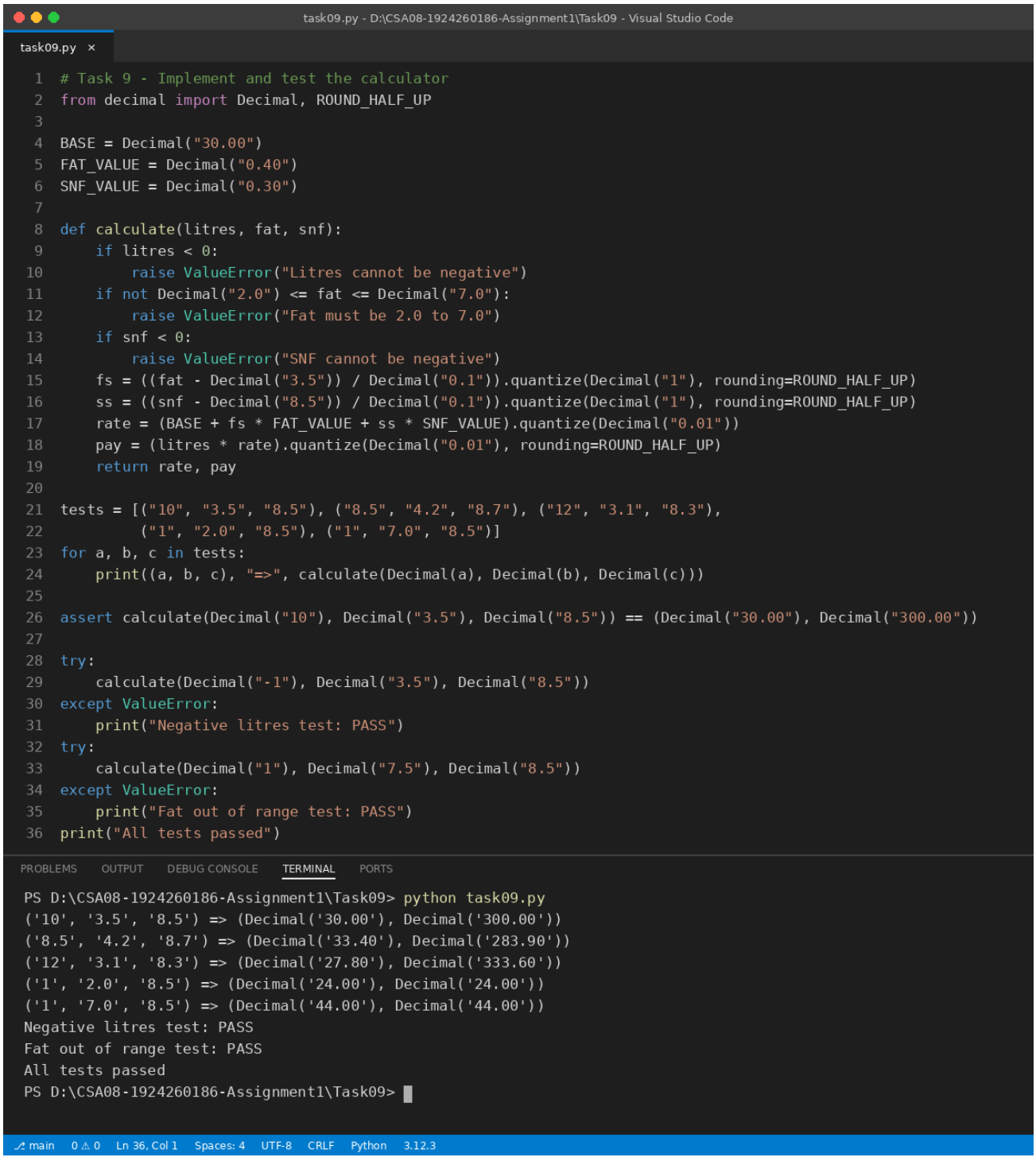
Task 8 - Plan the algorithm (hand-trace table)

Novia sri .A | 1924260186 | Python 3.12 | SIMATS Assignment 1



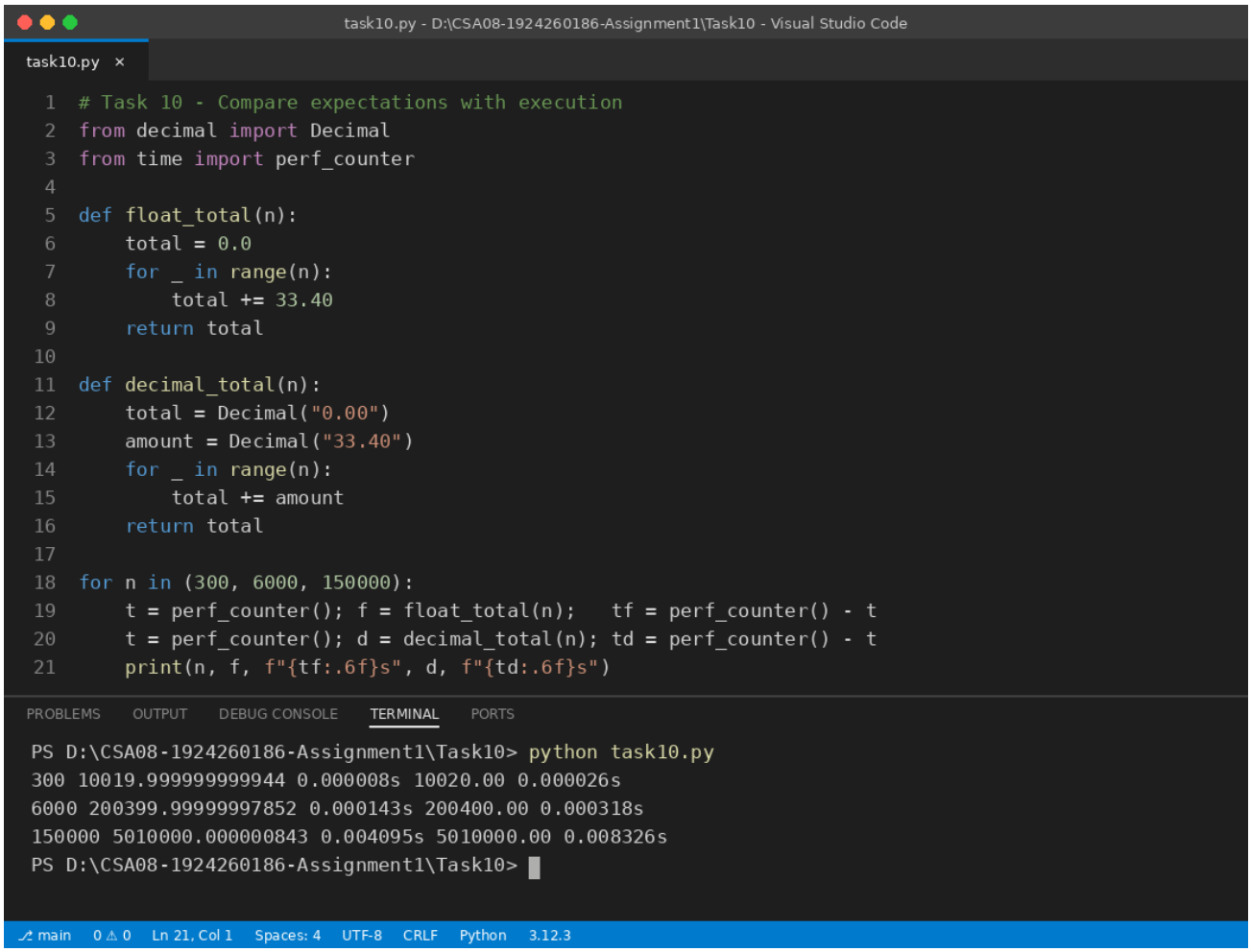
Task 9 - Implement and test the calculator

Novia sri .A | 1924260186 | Python 3.12 | SIMATS Assignment 1



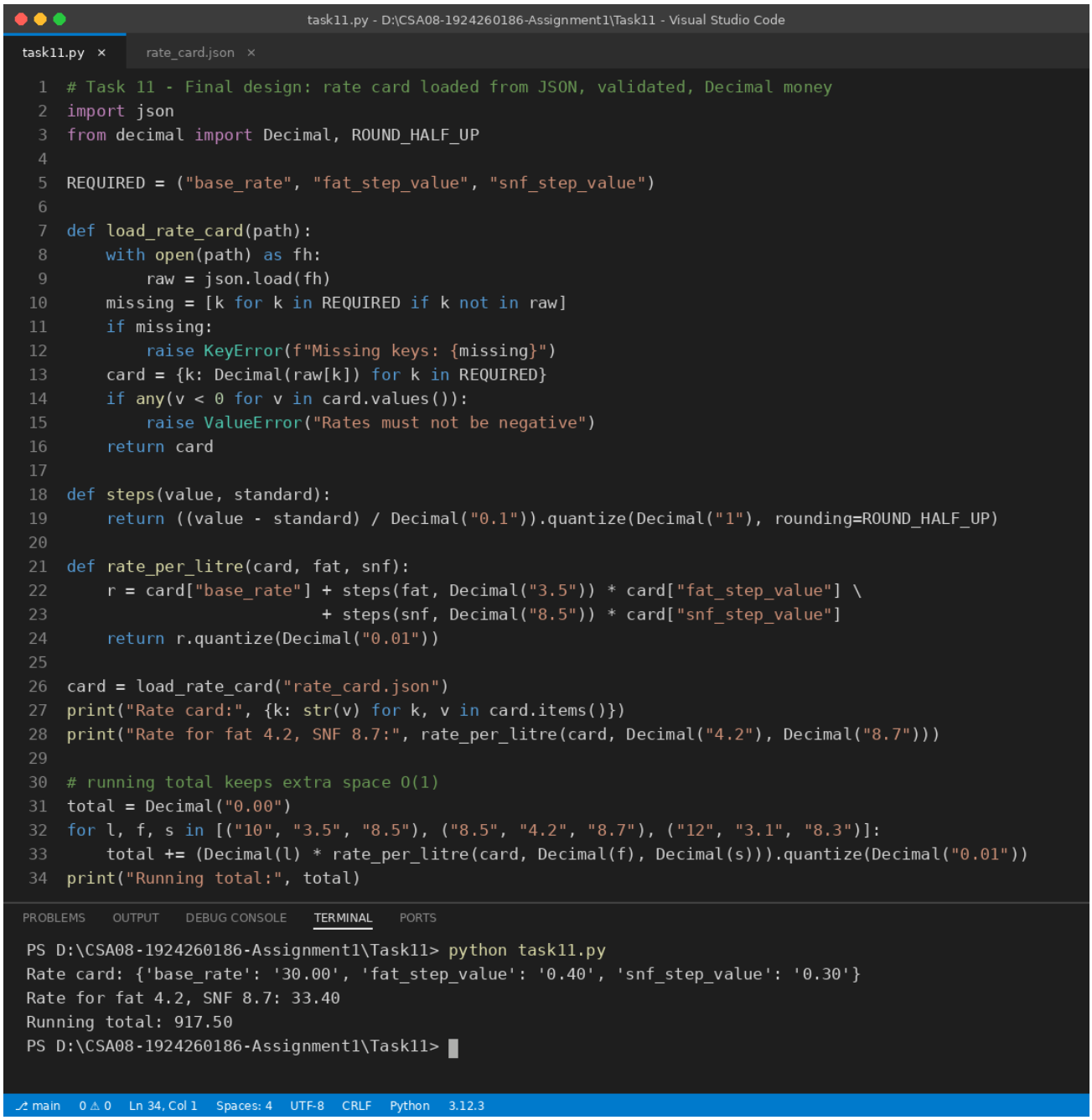
Task 10 - Compare expectations with execution

Novia sri .A | 1924260186 | Python 3.12 | SIMATS Assignment 1



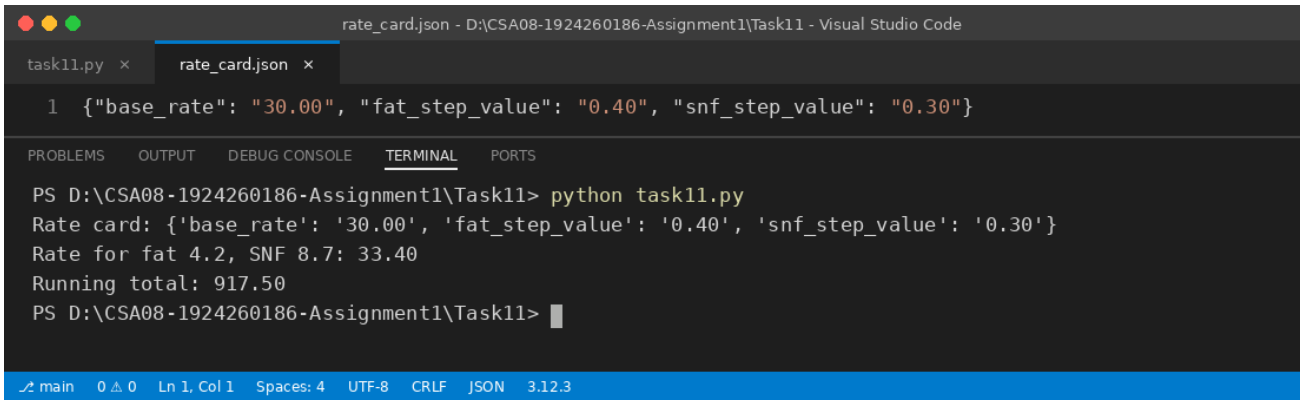
Task 11 - Final design with JSON rate card

Novia sri .A | 1924260186 | Python 3.12 | SIMATS Assignment 1



Task 11 - rate_card.json

Novia sri .A | 1924260186 | Python 3.12 | SIMATS Assignment 1



Task 12 - Seasonal rates and bonus

Novia sri .A | 1924260186 | Python 3.12 | SIMATS Assignment 1

